

NUMBER SYSTEMS CLASS NOTES

CS 6306 – Fundamentals Crash Course

Cyber Security Analytics and Malware Analysis for Practitioners

Course Description

A supervisory view of security vulnerabilities in information systems, applications and safe computing. Additionally, best practices to promote security standards and applications, management and analysis of security. Applications of data science techniques, preparation for adversarial attacks.

Course information

Course Title: Cyber Security Analytics and Malware Analysis for Practitioners

Section: 501

Course Term: Fall 2026

1. Why Number Systems Are in a Cybersecurity Course

I see that number systems are important because they show up constantly in the low-level information used during malware analysis. We see hexadecimal in memory addresses, executable-file data, and hex dumps, while binary helps us understand how computers store and manipulate information. I think learning these number systems first makes the later material easier to follow because malware runs on computer systems that ultimately process data in binary. We also need this foundation before working with concepts such as data types and bitwise operations. I believe this is the practical reason binary and hexadecimal matter so much in malware analysis: they help us understand what the computer is actually doing when we look at assembly instructions, disassembly, memory, executable files, and control-flow information.

2. The Core Idea: A Digit Gets Its Value From Its Position

Before talking about binary or hexadecimal, we need to understand positional notation. This is the idea that the same written digit can mean different amounts depending on where it appears.

2.1 Example using decimal

Take the decimal number 555. All three symbols are the digit 5, but they are not worth the same amount.

Position	Digit	Place value	What that 5 contributes
Hundreds	5	100	$5 \times 100 = 500$
Tens	5	10	$5 \times 10 = 50$
Ones	5	1	$5 \times 1 = 5$

The written symbol is the same, but position changes the amount. That same idea is used in binary, hexadecimal, and octal.

2.2 Why the far-right whole-number position is 1

For whole numbers, the far-right position is the base raised to zero power. A nonzero number raised to the zero power equals 1. So every system starts its whole-number place values with 1 on the far right.

System	Base	Far-right place	Next place left	Next place left
Binary	2	$2^0 = 1$	$2^1 = 2$	$2^2 = 4$
Octal	8	$8^0 = 1$	$8^1 = 8$	$8^2 = 64$
Decimal	10	$10^0 = 1$	$10^1 = 10$	$10^2 = 100$
Hexadecimal	16	$16^0 = 1$	$16^1 = 16$	$16^2 = 256$

Why do you always start with 1?

Because the rightmost whole-number position is base^0 , and $\text{base}^0 = 1$. We are not choosing 1 randomly.

3. Vocabulary You Need Before Conversions

3.1 Base and radix

“Base” and “radix” refer to the same underlying idea: which number system is being used. The base controls which single-digit symbols are available and how place values grow as we move left.

System	Base / radix	Single-digit symbols
Binary	2	0 and 1
Octal	8	0 through 7
Decimal	10	0 through 9
Hexadecimal	16	0 through 9 and A through F

3.2 Radix point

A radix point is the general name for the point that separates the whole-number part from the fractional part. In ordinary decimal writing, we usually call it a decimal point. We deal with integers, so you can imagine the radix point immediately after the last digit.

Example: 10110 can be thought of as 10110. because there is no fractional part after it.

3.3 Integer

An integer is a whole number with no fractional portion. Examples: 0, 5, 27, 1368. We examples are integer conversions.

3.4 Bit

A bit is one binary digit. A bit is either 0 or 1. So 10110 contains five bits.

3.5 Unsigned

Unsigned means the value is interpreted as zero or positive. For this lecture, an unsigned binary value is read using ordinary positive powers of 2. There is no negative interpretation being applied.

Does “unsigned” change the place values?

No. For the positive whole-number examples here, you still use the normal binary place values 1, 2, 4, 8, 16, and so on. “Unsigned” is telling you not to interpret a sign/negative value.

4. Decimal: Use the Familiar System to Understand the New Ones

Decimal is base 10. We use it every day. It uses ten single-digit symbols: 0 through 9.

4.1 Where 1, 10, 100, 1000 come from

Because decimal is base 10, each time you move one place left you multiply by 10.

1. Start at the far right with 1.
2. Move one place left: $1 \times 10 = 10$.
3. Move another place left: $10 \times 10 = 100$.
4. Move another place left: $100 \times 10 = 1000$.

Decimal place-value chain

1 -> 10 -> 100 -> 1000 -> 10000 ...

4.2 Example: 1368

The number 1368 has four positions. Starting at the far right, the decimal place values are 1, 10, 100, 1000.

Written digit	Why this place value?	Contribution
8	Far-right place = 1	$8 \times 1 = 8$
6	One place left = 10	$6 \times 10 = 60$
3	Next place = 100	$3 \times 100 = 300$
1	Next place = 1000	$1 \times 1000 = 1000$

Now add the contributions: $1000 + 300 + 60 + 8 = 1368$.

We also write this as $1 \times 10^3 + 3 \times 10^2 + 6 \times 10^1 + 8 \times 10^0$. That is simply another way to show the same place values.

Why this matters

Binary, hexadecimal, and octal use the same positional idea. Only the base changes, so the place values change.

5. Binary: Base 2

Binary is base 2. That means it only has two single-digit symbols: 0 and 1.

5.1 Why binary cannot use the digit 2

A base-2 system has only two allowed single-digit symbols. Those are 0 and 1. Once we have used 1, we cannot place a digit 2 in the same position. We must carry into a new position, just as decimal carries after 9.

5.2 Binary counting from 0 to 8, slowly

Decimal amount	Binary writing	Why
0	0	No value yet
1	1	One 1 in the ones place
2	10	Cannot write digit 2, so carry: one 2 and zero 1s
3	11	One 2 + one 1 = 3
4	100	New 4 place: one 4 + zero 2s + zero 1s
5	101	$4 + 1$
6	110	$4 + 2$
7	111	$4 + 2 + 1$
8	1000	New 8 place

Why does binary 10 mean decimal 2?

Because the positions are not decimal tens and ones. In binary, the two positions are 2 and 1. So 10 binary means $1 \times 2 + 0 \times 1 = 2$.

5.3 Binary labels in We

We note that binary can be labeled using forms such as 0b1010 or 1010b. The 0b or b is a label telling you the base. It does not add numeric value.

6. Binary Place Values: Where Every Number Comes From

This is the most important foundation for binary conversions. Do not memorize 1, 2, 4, 8, 16 as unrelated numbers. They come from repeatedly multiplying by the binary's base, which is 2.

6.1 Build the place values from scratch

Move from right	Calculation	Place value
Start	2^0	1
1 place left	1×2	2
2 places left	2×2	4
3 places left	4×2	8
4 places left	8×2	16
5 places left	16×2	32
6 places left	32×2	64
7 places left	64×2	128
8 places left	128×2	256
9 places left	256×2	512
10 places left	512×2	1024

If you forget a binary place value

Start at 1 on the far right and keep doubling as you move left. You can rebuild the entire list.

6.2 What does binary 1 actually mean?

A 1 means there is ONE copy of that place value. A 0 means there are ZERO copies of that place value. This is why people shorten the rule to "1 = use, 0 = skip."

Bit	Actual multiplication	Effect
1	$1 \times \text{place value}$	Keep/use that value
0	$0 \times \text{place value}$	Contributes 0, so skip it

7. Binary -> Decimal

Use this method when the starting number is binary and the question asks for its ordinary decimal value.

7.1 Slow method

1. Write the binary number.
2. Starting under the far-right bit, write 1.
3. Move left and double each time to create the place values.
4. For every bit that is 1, keep that place value.
5. For every bit that is 0, its contribution is 0.
6. Add the kept values.

7.2 0101 -> decimal

Step 1: 0101 has four bit positions. From right to left, the place values are 1, 2, 4, 8. Written left-to-right under 0101, that is 8, 4, 2, 1.

Step 2: Write the multiplication explicitly: $0 \times 8 + 1 \times 4 + 0 \times 2 + 1 \times 1$.

Step 3: $0 \times 8 = 0$, $1 \times 4 = 4$, $0 \times 2 = 0$, $1 \times 1 = 1$.

Step 4: Add the contributions: $0 + 4 + 0 + 1 = 5$.

Result

0101 binary = 5 decimal

Why can the leading 0 in 0101 be ignored?

That far-left 0 means $0 \times 8 = 0$. It does not add value. The number is still 5 whether written 0101 or 101.

7.3 101011 -> decimal

Step 1: There are seven bits, so build seven place values from the right: 1, 2, 4, 8, 16, 32, 64. Written under the bits left-to-right: 64, 32, 16, 8, 4, 2, 1.

Step 2: Match each bit to its place: 1,1,0,1,0,1,1.

Step 3: Keep the places under 1s: 64, 32, 8, 2, 1. Skip 16 and 4 because their bits are 0.

Step 4: Add: $64 + 32 + 8 + 2 + 1 = 107$.

Result

1101011 binary = 107 decimal

7.4 Reasonableness Check

If the leftmost 1 is in the 64 place, the value must be at least 64. The next power of 2 is 128, so a seven-bit value beginning with 1 must be less than 128. Since 107 is between 64 and 127, the result is reasonable.

8. The 0-15 Binary Map: Build It Instead of Treating It as Random

We highlight decimal 0 through 15 because four binary bits can represent exactly those sixteen values. These same sixteen values are also exactly the values of one hexadecimal digit.

8.1 Why four bits stop at 15

A four-bit group has place values 8, 4, 2, 1. The largest possible four-bit pattern is 1111, which uses every place: $8 + 4 + 2 + 1 = 15$. The smallest is 0000 = 0. Therefore four bits cover every whole number from 0 through 15.

4-bit binary	What the 1s use	Decimal amount	Hex writing
0000	0	0	0
0001	1	1	1
0010	2	2	2
0011	2 + 1	3	3
0100	4	4	4
0101	4 + 1	5	5
0110	4 + 2	6	6
0111	4 + 2 + 1	7	7
1000	8	8	8
1001	8 + 1	9	9
1010	8 + 2	10	A
1011	8 + 2 + 1	11	B
1100	8 + 4	12	C
1101	8 + 4 + 1	13	D
1110	8 + 4 + 2	14	E
1111	8 + 4 + 2 + 1	15	F

How to read any row

Example: 1101 uses $8 + 4 + 1 = 13$. Decimal 13 is written D in hexadecimal. Therefore 1101 binary, 13 decimal, and D hexadecimal are three writings of the same amount.

9. Decimal -> Binary

Now the direction reverses. Instead of being given bits and finding their decimal total, you are given a decimal total and must decide which binary place values are needed to build it.

9.1 What The Method Is Doing

We repeatedly subtract the largest power of 2 that fits without making the remainder negative. If a place value is used, write 1 in that position. If it is skipped, write 0.

Why do skipped places still need a 0?

Because position matters. A 0 holds the place so the 1s stay in the correct positions. Removing a middle 0 would move later bits into different place values and change the number.

9.2 50 -> binary

Step 1: List nearby powers of 2: 1, 2, 4, 8, 16, 32, 64. 64 is too large, so start with 32.

Step 2: $50 - 32 = 18$. We used the 32 place, so that position will be 1.

Step 3: 16 fits into 18. $18 - 16 = 2$. The 16 position will be 1.

Step 4: 8 does not fit into 2, so the 8 position is 0. 4 also does not fit, so the 4 position is 0.

Step 5: 2 fits exactly. $2 - 2 = 0$. The 2 position is 1.

Step 6: We did not need the 1 place, so it is 0.

Step 7: Read the positions 32,16,8,4,2,1 as 1,1,0,0,1,0.

Result

50 decimal = 110010 binary

9.3 76 -> binary

Step 1: Nearby powers are 1,2,4,8,16,32,64,128. 128 is too large; 64 fits.

Step 2: $76 - 64 = 12$, so the 64 place is 1.

Step 3: 32 is too large for 12, so 32 place = 0. 16 is also too large, so 16 place = 0.

Step 4: 8 fits: $12 - 8 = 4$, so 8 place = 1.

Step 5: 4 fits: $4 - 4 = 0$, so 4 place = 1.

Step 6: 2 and 1 are not needed after the remainder reaches 0, so both are 0.

Step 7: Read 64,32,16,8,4,2,1 as 1,0,0,1,1,0,0.

Result

76 decimal = 1001100 binary

10. Hexadecimal: Base 16

Hexadecimal is base 16. One hexadecimal digit must be able to represent sixteen possible values before carrying into a new position.

10.1 Why hexadecimal needs letters

The ordinary decimal digit symbols 0 through 9 give us only ten single-digit symbols. Hexadecimal needs sixteen. Six additional symbols are needed for the values that come after 9, so the standard notation uses A through F.

Hex symbol	Decimal amount it represents	Why
0-9	Same amounts 0-9	Those symbols already exist
A	10	First value after 9
B	11	Next value
C	12	Next value
D	13	Next value
E	14	Next value
F	15	Last one-digit hex value

10.2 Are A-F mathematically forced?

The LETTER CHOICE is a notation convention: people agreed to use A-F. But once the hexadecimal is defined, the values are fixed in order: A=10, B=11, C=12, D=13, E=14, F=15.

10.3 Hexadecimal counting, slowly

A single hexadecimal digit counts through all sixteen values:

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F

After F, there is no unused one-digit symbol left. So hexadecimal carries into a new position.

Why does F + 1 become hex 10?

F means decimal 15. Adding 1 gives decimal 16. In base 16, one full group of 16 is written as 1 in the 16s place and 0 in the 1s place: hex 10. So hex 10 = decimal 16, not decimal 10.

10.4 Hex labels

We mention 0x as a hexadecimal prefix and h as a hexadecimal suffix. These are labels indicating the base.

11. Why Binary and Hexadecimal Match in Groups of 4

This is one of the most important “why” explanations in We.

11.1 Start with four binary places

Four binary positions are worth 8, 4, 2, 1. Those four positions can produce values from 0 through 15.

The number of possible four-bit patterns is $2 \times 2 \times 2 \times 2 = 16$, because each of the four positions has two choices: 0 or 1.

This is also written as $2^4 = 16$.

11.2 Compare that with hexadecimal

One hexadecimal digit also has sixteen possible values: 0 through F. Therefore one hexadecimal digit can represent exactly the same range of values as four binary bits.

This is why

Binary $\leftrightarrow$ hexadecimal uses groups of 4. It is not a random grouping rule; 4 binary bits have exactly 16 possibilities, matching base 16.

12. Deriving the Entire Binary $\leftrightarrow$ Hexadecimal Chart

Whenever a mapping looks random, convert the four bits to their ordinary decimal amount first. Then translate that amount to the hexadecimal symbol.

12.1 The four binary place values are 8, 4, 2, 1

We get those values by starting with 1 on the right and doubling left: 1, 2, 4, 8. Written left-to-right under a four-bit group, they are 8, 4, 2, 1.

0000 $\rightarrow$ 0 = 0. Hexadecimal uses the same digit 0 for that amount.

0001 $\rightarrow$ 1 = 1. Hexadecimal uses the same digit 1 for that amount.

0010 -> $2 = 2$. Hexadecimal uses the same digit 2 for that amount.

0011 -> $2 + 1 = 3$. Hexadecimal uses the same digit 3 for that amount.

0100 -> $4 = 4$. Hexadecimal uses the same digit 4 for that amount.

0101 -> $4 + 1 = 5$. Hexadecimal uses the same digit 5 for that amount.

0110 -> $4 + 2 = 6$. Hexadecimal uses the same digit 6 for that amount.

0111 -> $4 + 2 + 1 = 7$. Hexadecimal uses the same digit 7 for that amount.

1000 -> $8 = 8$. Hexadecimal uses the same digit 8 for that amount.

1001 -> $8 + 1 = 9$. Hexadecimal uses the same digit 9 for that amount.

1010 -> $8 + 2 = 10$. Hexadecimal writes decimal 10 as A.

1011 -> $8 + 2 + 1 = 11$. Hexadecimal writes decimal 11 as B.

1100 -> $8 + 4 = 12$. Hexadecimal writes decimal 12 as C.

1101 -> $8 + 4 + 1 = 13$. Hexadecimal writes decimal 13 as D.

1110 -> $8 + 4 + 2 = 14$. Hexadecimal writes decimal 14 as E.

1111 -> $8 + 4 + 2 + 1 = 15$. Hexadecimal writes decimal 15 as F.

Most important examples

1010 -> $8+2 = 10$ -> A. 1011 -> $8+2+1 = 11$ -> B. 1100 -> $8+4 = 12$ -> C. 1101 -> $8+4+1 = 13$ -> D. 1110 -> $8+4+2 = 14$ -> E. 1111 -> $8+4+2+1 = 15$ -> F.

13. Hexadecimal -> Decimal

Hexadecimal is base 16, so its place values grow by multiplying by 16 as you move left.

13.1 Where 1, 16, 256, 4096 come from

1. Far-right place: $16^0 = 1$.
2. One place left: $1 \times 16 = 16$.
3. Next place left: $16 \times 16 = 256$.
4. Next place left: $256 \times 16 = 4096$.

Hex place values

... 4096 | 256 | 16 | 1

13.2 B2F -> decimal

We use B2F. We will explain every value before multiplying anything.

1. B is a hexadecimal digit. In the hex sequence, B represents decimal 11.
2. 2 already represents decimal 2.
3. F represents decimal 15.
4. There are three hex positions. From right to left the place values are 1, 16, 256. Therefore B is in the 256 place, 2 is in the 16 place, and F is in the 1 place.
5. Multiply each digit amount by its place: 11×256 , 2×16 , 15×1 .
6. $11 \times 256 = 2816$. $2 \times 16 = 32$. $15 \times 1 = 15$.
7. Add: $2816 + 32 + 15 = 2863$.

Final

B2F hexadecimal = 2863 decimal

Why was B multiplied by 256?

Because B is the third hex digit from the right, and the third base-16 place is $16^2 = 256$. The 256 was produced by the base and the position, not chosen randomly.

14. Binary -> Hexadecimal

Once you understand the 4-bit relationship, this conversion becomes a grouping problem.

14.1 Why grouping starts from the RIGHT

The far right is where the lowest place values begin. A hexadecimal digit replaces one block of four binary place positions. Grouping from the right keeps the place positions aligned correctly.

14.2 What if the leftmost group is too short?

You may add zeros only on the far left of the incomplete group. These are temporary leading zeros. They contribute no value but let you complete a four-bit group.

Does adding a leading zero change the number?

No. A leading zero contributes 0 x its place value. For example 1 and 0001 have the same numerical amount.

14.3 Procedure

1. Start from the right edge.
2. Split into groups of 4 bits.
3. If the leftmost group has fewer than 4 bits, add zeros on its LEFT until it has 4.
4. Convert each group by using the 8,4,2,1 values or the derived chart.
5. Join the hexadecimal digits.
6. Do not keep unnecessary leading zeros that were added only for grouping.

14.4 Original lecture example: 100011111001 -> hex

Step 1: Group from the right: 1000 | 1111 | 1001.

Step 2: 1000 uses only the 8 place, so its value is 8 -> hex 8.

Step 3: 1111 uses $8+4+2+1 = 15$ -> hex F.

Step 4: 1001 uses $8+1 = 9$ -> hex 9.

Step 5: Join the three hex digits in the same order.

Result

8F9

14.5 111101111000010 -> hex

Step 1: There are 15 bits. Grouping from the right leaves the far-left group short, so pad only on the left: 0111 | 1011 | 1100 | 0010.

Step 2: $0111 = 4+2+1 = 7 \rightarrow$ hex 7.

Step 3: $1011 = 8+2+1 = 11 \rightarrow$ hex B.

Step 4: $1100 = 8+4 = 12 \rightarrow$ hex C.

Step 5: $0010 = 2 \rightarrow$ hex 2.

Step 6: Join: 7BC2. The temporary leading 0 was only there to complete the group and is not written in the final answer.

Result

7BC2

15. Hexadecimal -> Binary

This is the reverse. Each hexadecimal digit represents one value from 0 through 15, and that value has one four-bit binary pattern.

15.1 Why each hex digit gets exactly four bits

One hex digit has sixteen possible values. Four binary bits also have sixteen possible patterns. So keep a full four-bit group for each hex digit while converting.

15.2 How to derive a letter if you forget the chart

Example: D. D means decimal 13. Ask which values among 8,4,2,1 make 13: $8+4+1 = 13$. So the used/skipped pattern is 1,1,0,1 $\rightarrow$ 1101.

15.3 Procedure

1. Take one hexadecimal digit at a time.
2. Translate the hex digit to its decimal amount if needed (A=10 through F=15).
3. Use 8,4,2,1 to build a four-bit pattern for that amount.
4. Keep exactly four bits for each hex digit while assembling the answer.
5. Join the groups.
6. At the very end, if plain binary is requested, remove only unnecessary zeros at the FAR LEFT of the entire result.

15.4 E14 -> binary

Step 1: E means decimal 14. $14 = 8+4+2$, so the four-bit pattern is 1110.

Step 2: Hex digit 1 must still occupy a four-bit group: 0001.

Step 3: Hex digit 4 becomes 0100 because only the 4 place is used.

Step 4: Join the groups: 1110 0001 0100.

Result

111000010100

15.5 Original lecture example: 2DA0 -> binary

Step 1: 2 $\rightarrow$ decimal 2 $\rightarrow$ four-bit 0010.

Step 2: D $\rightarrow$ decimal 13 $\rightarrow 8+4+1 \rightarrow$ 1101.

Step 3: A $\rightarrow$ decimal 10 $\rightarrow 8+2 \rightarrow$ 1010.

Step 4: 0 $\rightarrow$ 0000. This group must stay because it represents an actual hexadecimal digit 0.

Step 5: Join: 0010 1101 1010 0000.

Step 6: Only the unnecessary zeros at the far-left edge may be removed, giving 10110110100000.

Result

10110110100000

Why can I remove the first zeros but not the 0000 at the end?

The first zeros are leading zeros before the first meaningful 1, so they add no positional information. The ending 0000 is the binary representation of the actual final hexadecimal digit 0, so removing it would change the number.

16. Octal: Base 8

The last number system introduced in We is octal. Octal is base 8, so it uses exactly eight single-digit symbols: 0 through 7.

16.1 Why octal stops at 7

A base-8 system has eight one-digit values. Counting begins 0,1,2,3,4,5,6,7. After 7, there is no additional one-digit symbol, so the system carries into another position.

16.2 Why exactly 3 binary bits match one octal digit

Three binary positions are worth 4,2,1. The smallest pattern is 000 = 0. The largest is 111 = 4+2+1 = 7. That gives exactly the eight values 0 through 7, matching the eight octal digits.

The same idea can be written as $2^3 = 8$.

This is why

Binary $\leftrightarrow$ octal uses groups of 3. It is not a random rule; three bits provide exactly eight patterns, matching base 8.

16.3 Derive the entire 3-bit map

000 $\rightarrow$ 0 = 0 $\rightarrow$ octal digit 0

001 $\rightarrow$ 1 = 1 $\rightarrow$ octal digit 1

010 $\rightarrow$ 2 = 2 $\rightarrow$ octal digit 2

011 $\rightarrow$ 2 + 1 = 3 $\rightarrow$ octal digit 3

100 $\rightarrow$ 4 = 4 $\rightarrow$ octal digit 4

101 $\rightarrow$ 4 + 1 = 5 $\rightarrow$ octal digit 5

110 $\rightarrow$ 4 + 2 = 6 $\rightarrow$ octal digit 6

111 $\rightarrow$ 4 + 2 + 1 = 7 $\rightarrow$ octal digit 7

16.4 Binary $\rightarrow$ octal

1. Group binary digits from the RIGHT in groups of 3.
2. If the leftmost group is short, add leading zeros to its left.
3. For each group, use the place values 4,2,1 to determine the octal digit.

16.5 Octal -> binary

1. Take each octal digit separately.
2. Write that value as exactly three binary bits using 4,2,1.
3. Join the groups.
4. Remove only unnecessary far-left zeros if a plain binary answer is requested.

17. Leading Zeros, Internal Zeros, and Temporary Padding

17.1 Leading zeros

A leading zero is a zero before the first meaningful nonzero digit. It contributes zero and does not change the amount.

Writing	Same value	Why
0101	101	The leftmost 0 contributes 0
0001	1	All extra left zeros contribute 0
0010	10	The extra far-left zeros do not move the meaningful positions

17.2 Internal zeros

A zero inside the number is different. It marks that a particular place value is not used while preserving the positions of the digits around it. Removing an internal zero changes the place values and therefore changes the number.

Can I delete every 0 because 0 means “skip”?

No. “Skip” means that a specific position contributes no value during a calculation. The 0 still has to remain in the written binary number so the other digits stay in their correct positions.

17.3 Temporary padding

Padding is the act of adding leading zeros only to make a complete group for conversion. Binary -> hex may require four-bit groups; binary -> octal may require three-bit groups. Padding is a grouping aid, not a change in numerical value.

18. Prefixes, Suffixes, and Answer Formatting

We mention notation that can label the base. These labels are useful in technical writing.

Example	Meaning
0b1010	0b labels the value as binary
1010b	b suffix labels binary in the convention shown in We
0x2F	0x labels hexadecimal
2Fh	h suffix labels hexadecimal

19. How to Decide What Method to Use

Before doing any arithmetic, identify the direction of the conversion.

You are given	You need	What to think
Binary	Decimal	Place values: start at 1 on the right, double left, use 1s, skip 0s, add
Decimal	Binary	Which powers of 2 build this number? Used=1, skipped=0
Binary	Hexadecimal	Group from right in 4s; each group uses 8,4,2,1
Hexadecimal	Binary	Each hex digit becomes 4 bits; derive letters through decimal if needed
Hexadecimal	Decimal	Translate A-F first, then use place values 1,16,256,4096...
Binary	Octal	Group from right in 3s; each group uses 4,2,1
Octal	Binary	Each octal digit becomes 3 bits using 4,2,1

If you feel lost

Do not start calculating yet. First say out loud: "I am converting FROM ____ TO ____." Then pick the matching method.

20. What to Do Whenever a Mapping Feels Random

This section is specifically for the problem you described: seeing one symbol suddenly assigned to another symbol.

20.1 Use decimal as the bridge

Decimal is familiar, so use it as an intermediate checkpoint whenever binary and hexadecimal feel disconnected.

Looks random	Break it into understandable steps
1010 = A	1010 binary -> 8+2 -> decimal 10 -> hex A
1011 = B	1011 -> 8+2+1 -> decimal 11 -> hex B
1100 = C	1100 -> 8+4 -> decimal 12 -> hex C
1101 = D	1101 -> 8+4+1 -> decimal 13 -> hex D
1110 = E	1110 -> 8+4+2 -> decimal 14 -> hex E
1111 = F	1111 -> 8+4+2+1 -> decimal 15 -> hex F
E = 1110	E -> decimal 14 -> 8+4+2 -> 1110
D = 1101	D -> decimal 13 -> 8+4+1 -> 1101

Key mental model

Different number systems are different ways of WRITING the same quantity. Decimal 10, binary 1010, and hexadecimal A are not three different amounts. They are three notations for the same amount.